Worksheet - 3

- **Aim(A):** To implement classification algorithms—Logistic Regression, Decision Tree, and k-Nearest Neighbour's (k-NN)—on a suitable dataset and evaluate their performance using confusion matrix, precision, recall, and F1-score.

- **Objective(A):** To implement Logistic Regression and Polynomial Regression algorithms on a suitable dataset and evaluate their performance using confusion matrix, precision, recall, and F1-score, in order to understand and compare the effectiveness of regression-based machine learning techniques for prediction and classification tasks.

- **Code(A):**

LOGISTIC REGRESSION

```
print("=" * 65)
print("PART 1: LOGISTIC REGRESSION CLASSIFICATION")
print("=" * 65)
target_col = 'evidence level - score. 0 = no evidence, 1,2 = slight, 3 = conflicting , 4 = promising, 5 = good, 6 = strong '
df['Target'] = (df[target_col] >= 4).astype(int)
features_lr = [
'popularity',
'number of studies examined',
'number of citations',
'Cochrane systematic review',
'Other International review board or metastudy',
'No. of individual studies / reviews'
]
X_lr = df[features_lr]
y_lr = df['Target']
X_train_lr, X_test_lr, y_train_lr, y_test_lr = train_test_split(
X_lr, y_lr, test_size=0.2, random_state=42, stratify=y_lr
)
lr_model = LogisticRegression(max_iter=1000)
lr_model.fit(X_train_lr, y_train_lr)
y_pred_lr = lr_model.predict(X_test_lr)
cm_lr = confusion_matrix(y_test_lr, y_pred_lr)
acc_lr = accuracy_score(y_test_lr, y_pred_lr)
prec_lr = precision_score(y_test_lr, y_pred_lr, zero_division=0)
rec_lr = recall_score(y_test_lr, y_pred_lr, zero_division=0)
f1_lr = f1_score(y_test_lr, y_pred_lr, zero_division=0)
print("\nLogistic Regression Results")
print("Confusion Matrix:")
print(cm_lr)
print(f"\nAccuracy  : {acc_lr:.4f}")
print(f"Precision : {prec_lr:.4f}")
print("\nClassification Report:")
print(classification_report(y_test_lr, y_pred_lr, zero_division=0))
```

Saurav Bhuria (25MCA20133)
25MCA-6 (B)

```
Logistic Regression Results
Confusion Matrix:
[[17  0]
 [ 2  1]]

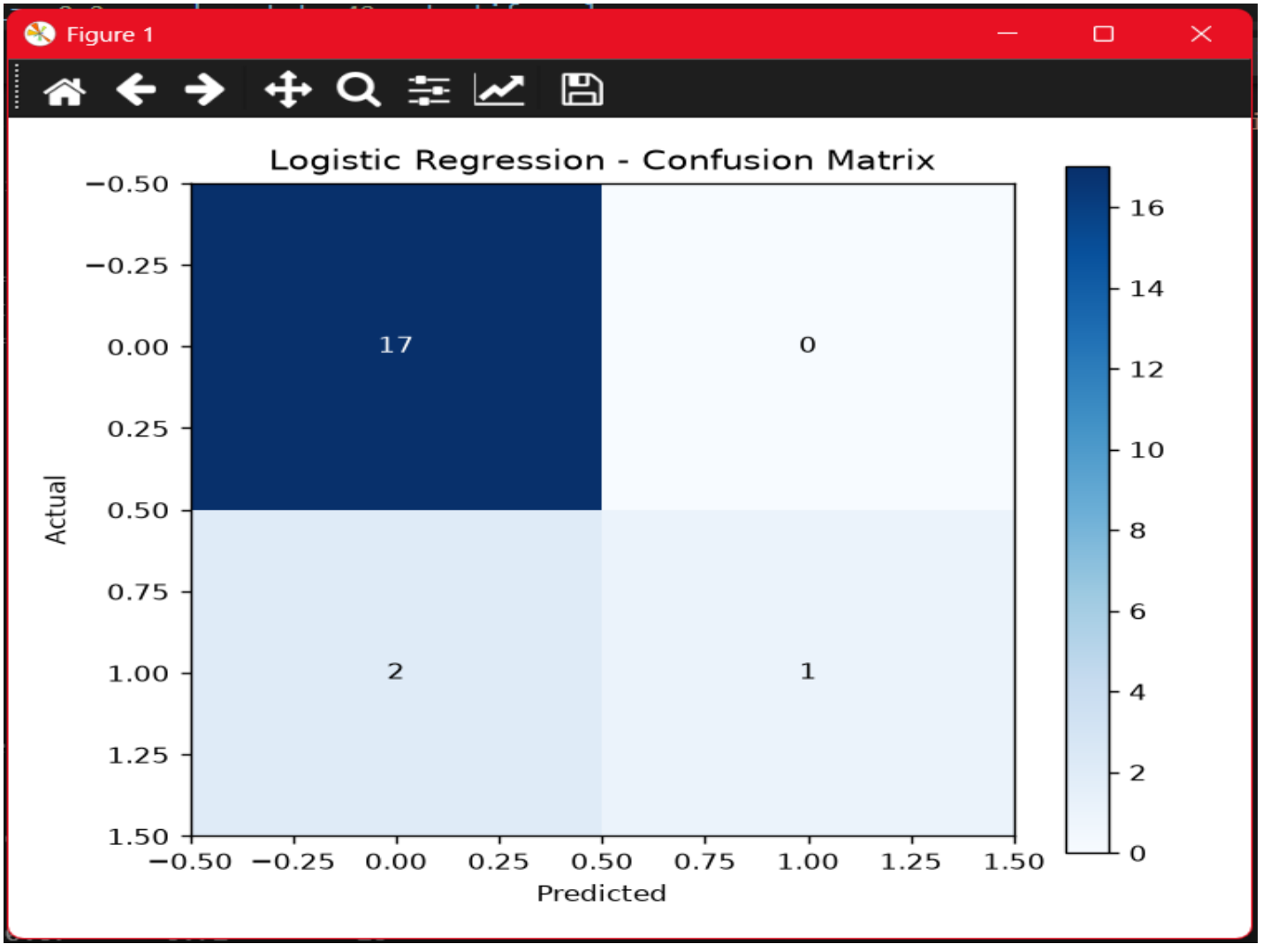
Accuracy  : 0.9000
Precision : 1.0000
Recall    : 0.3333
F1-Score  : 0.5000

Classification Report:
              precision    recall  f1-score   support

           0       0.89      1.00      0.94        17
           1       1.00      0.33      0.50         3

    accuracy                           0.90        20
   macro avg       0.95      0.67      0.72        20
weighted avg       0.91      0.90      0.88        20

Saved: c:\Users\HP\OneDrive\Desktop\25 MCA CU\25MCA20133\3rd Sem 2026\25CAP-702 Machine Learning Lab\Experiment 3A\logistic_regression_confusion_matrix.png
```



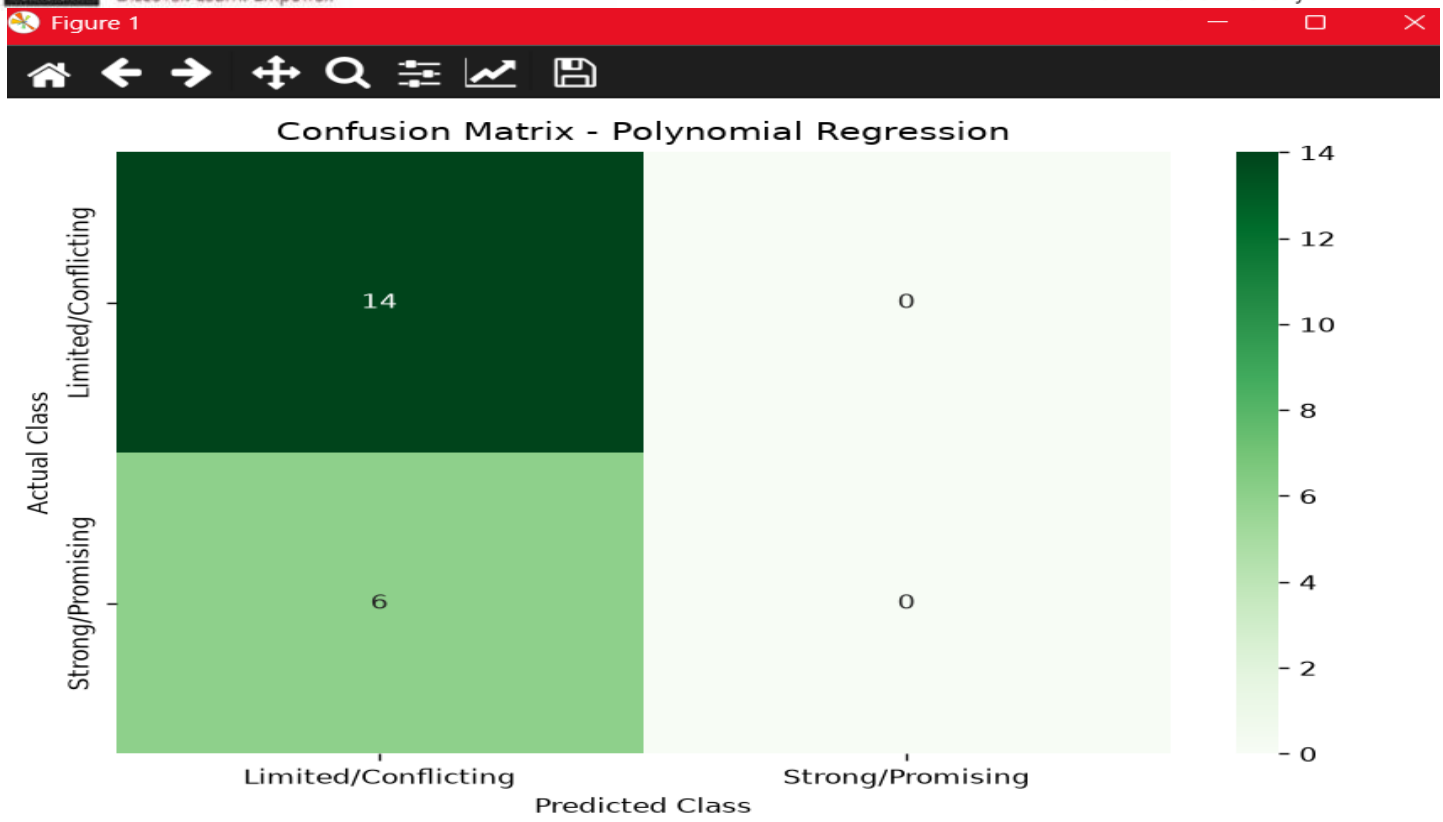
POLYNOMIAL REGRESSION

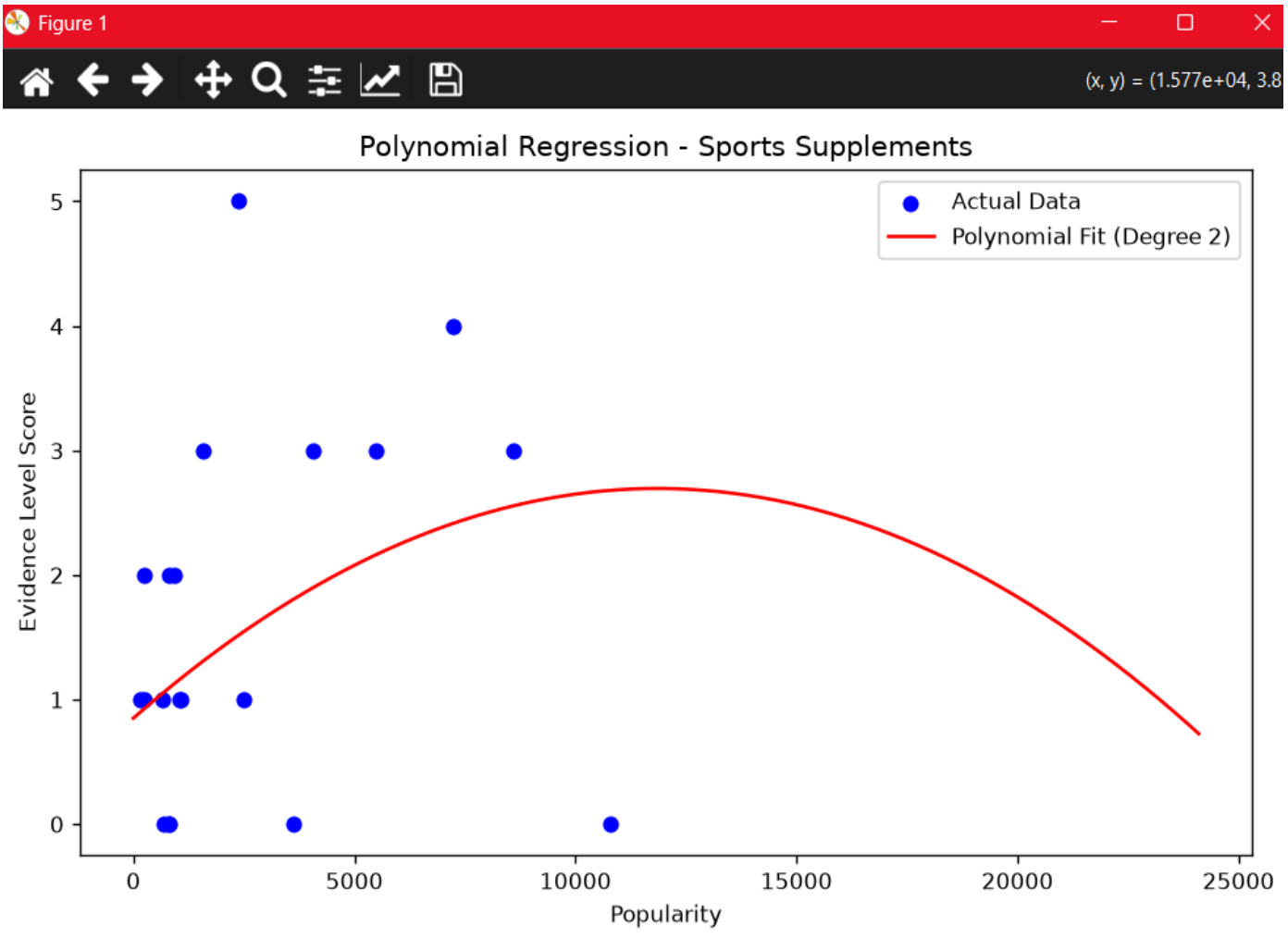
```
print("\n" + "=" * 65)
print("PART 2: POLYNOMIAL REGRESSION CLASSIFICATION")
print("=" * 65)
score_column = df.columns[2]
data_poly = df[['popularity', score_column]].copy()
data_poly.columns = ['popularity', 'evidence_score']
data_poly['popularity'] = pd.to_numeric(data_poly['popularity'], errors='coerce')
data_poly['evidence_score'] = pd.to_numeric(data_poly['evidence_score'], errors='coerce')
data_poly = data_poly.dropna()
print(f"Cleaned records for Polynomial Regression: {len(data_poly)}")
X_p = data_poly[['popularity']]
y_p = data_poly['evidence_score']
X_train_p, X_test_p, y_train_p, y_test_p = train_test_split(
    X_p, y_p, test_size=0.20, random_state=42
)
print(f"Training samples: {len(X_train_p)}, Testing samples: {len(X_test_p)}")
poly_model = make_pipeline(
    PolynomialFeatures(degree=2),
    LinearRegression()
)
poly_model.fit(X_train_p, y_train_p)
y_pred_p = poly_model.predict(X_test_p)
mae_p = mean_absolute_error(y_test_p, y_pred_p)
mse_p = mean_squared_error(y_test_p, y_pred_p)
rmse_p = np.sqrt(mse_p)
r2_p = r2_score(y_test_p, y_pred_p)
print("\nREGRESSION PERFORMANCE")
print(f"MAE  : {mae_p:.4f}")
```

```
PART 2: POLYNOMIAL REGRESSION CLASSIFICATION
=================================================================
Cleaned records for Polynomial Regression: 96
Training samples: 76, Testing samples: 20

REGRESSION PERFORMANCE
MAE  : 1.0413
MSE  : 1.8339
RMSE : 1.3542
R?   : 0.0955

CLASSIFICATION PERFORMANCE (Threshold >= 3)
Confusion Matrix:
[[14  0]
 [ 6  0]]
Accuracy  : 0.7000
Precision : 0.0000
Recall    : 0.0000
F1-Score  : 0.0000
Saved: c:\Users\HP\OneDrive\Desktop\25 MCA CU\25MCA20133\3rd Sem 2026\25CAP-702 Machine Learning Lab\Experiment 3A\polynomial_regression_confusion_matrix.png
Saved: c:\Users\HP\OneDrive\Desktop\25 MCA CU\25MCA20133\3rd Sem 2026\25CAP-702 Machine Learning Lab\Experiment 3A\polynomial_regression_curve.png
```





Saurav Bhuria (25MCA20133)
25MCA-6 (B)

```
Saved: c:\Users\HP\OneDrive\Desktop\25 MCA CU\25MCA20133\3rd Sem 2026\25CAP-702 Machine Learning Lab\Experiment
3A\polynomial_regression_curve.png

================================================================
EXPERIMENT 3A SUMMARY
================================================================
   Metric Logistic Regression Polynomial Regression
 Accuracy              90.00%                70.00%
Precision             100.00%                 0.00%
   Recall              33.33%                 0.00%
 F1-Score              50.00%                 0.00%

[Done] exited with code=0 in 193.843 seconds
```

- **Learning outcomes(A):**
  1. Understood and implement Polynomial Regression for predicting continuous values.
  2. Applied data preprocessing and prepare a suitable dataset for model training.
  3. Divided the dataset into training and testing sets for model evaluation.
  4. Transformed input features into polynomial features and train a regression model.

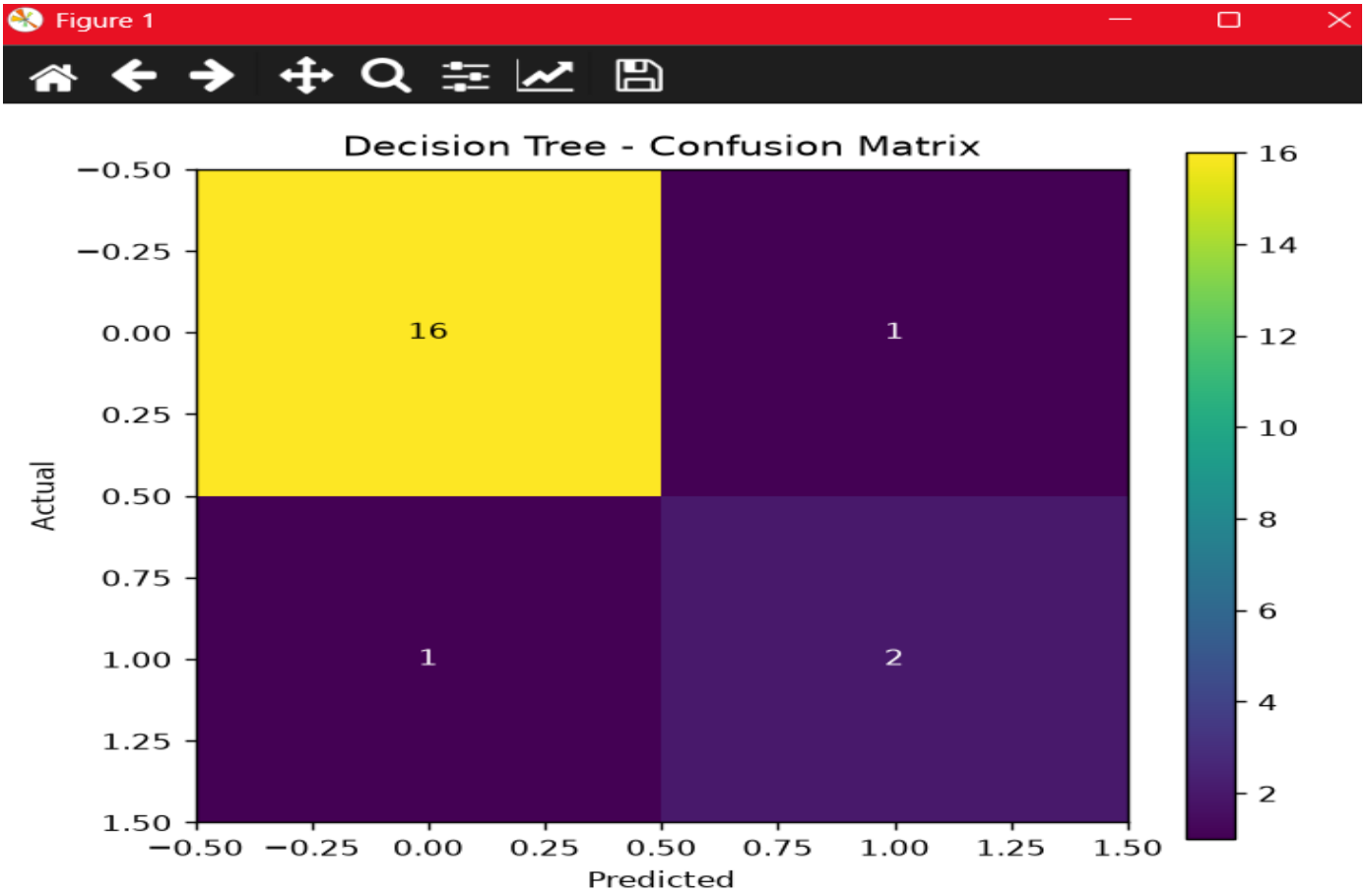
- **Aim(B):** Implement classification algorithms— Decision Tree, and k-NN— on a suitable dataset and evaluate their performance using confusion matrix, precision, recall, and F1-score.

- **Objective(B):** To implement Decision Tree and k-Nearest Neighbour's (k-NN) classification algorithms on a suitable dataset and evaluate their performance using confusion matrix, precision, recall, and F1-score to compare their effectiveness in classification tasks.

- **Code(B): Decision Tree**

```
import os
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.metrics import confusion_matrix, precision_score, recall_score, f1_score, classification_report
csv_filename = "Sports Supplements.csv"
if os.path.exists(csv_filename):
csv_path = csv_filename
elif os.path.exists(os.path.join(os.path.dirname(os.path.abspath(__file__)), csv_filename)):
csv_path = os.path.join(os.path.dirname(os.path.abspath(__file__)), csv_filename)
elif os.path.exists("OneDrive/Desktop/Machine Learning/Sports Supplements.csv"):
csv_path = "OneDrive/Desktop/Machine Learning/Sports Supplements.csv"
else:
csv_path = r"C:\Users\HP\OneDrive\Desktop\Machine Learning\Sports Supplements.csv"
df = pd.read_csv(csv_path)
print(f"Dataset loaded successfully from: {csv_path}")
print(f"Dataset shape: {df.shape}")
target = 'evidence level - score. 0 = no evidence, 1,2 = slight, 3 = conflicting , 4 = promising, 5 = good, 6 =
  strong '
df['Target'] = (df[target] >= 4).astype(int)
```

```
features = [
'popularity',
'number of studies examined',
'number of citations',
'Cochrane systematic review',
'Other International review board or metastudy',
'No. of individual studies / reviews'
```




```
plt.tight_layout()
cm_plot_path = os.path.join(os.path.dirname(os.path.abspath(__file__)),
"decision_tree_confusion_matrix.png")
plt.savefig(cm_plot_path, dpi=300)
print(f"Confusion matrix plot saved to: {cm_plot_path}")
plt.show()
plt.figure(figsize=(15, 8))
plot_tree(
model,
feature_names=features,
class_names=["Low Evidence", "Good Evidence"],
filled=True,
rounded=True,
fontsize=9
)
plt.title("Decision Tree Visualization")
```

```
tree_plot_path = os.path.join(os.path.dirname(os.path.abspath(__file__)), "decision_tree_structure.png")
plt.savefig(tree_plot_path, dpi=300, bbox_inches='tight')
print(f"Decision tree structure saved to: {tree_plot_path}")
plt.show()
```

```
Problems  Output  Debug Console  Terminal  Ports   Filter (e.g. text, !excludeText, t...   Code

[Done] exited with code=0 in 7.809 seconds

[Running] python -u "c:\Users\HP\OneDrive\Desktop\25 MCA CU\25MCA20133\3rd Sem 2026\25CAP-702 Machine Learning Lab\Experiment 3B\decision_tree.py"
Dataset loaded successfully from: Sports Supplements.csv
Dataset shape: (96, 46)

========================================
Decision Tree Results
========================================
Confusion Matrix:
[[16  1]
 [ 1  2]]

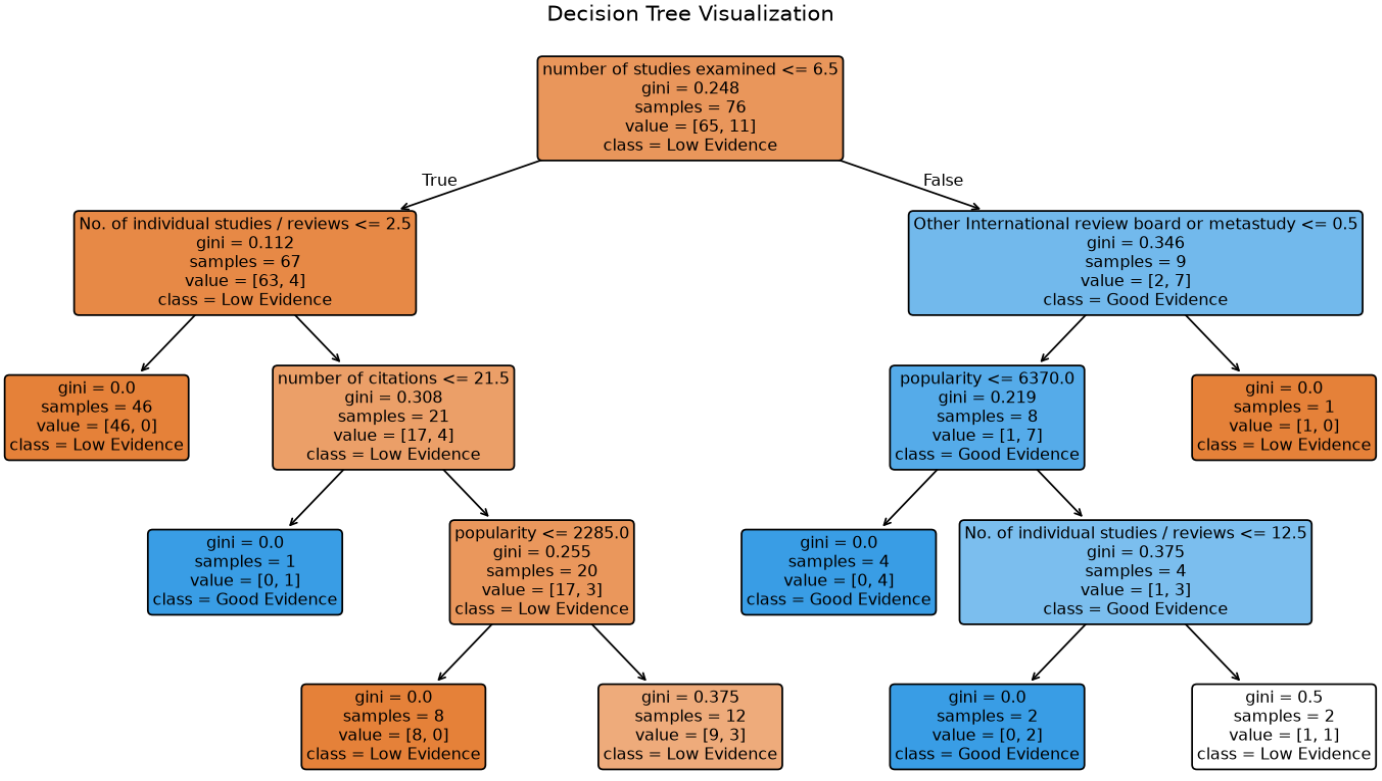
Precision: 0.6666666666666666
Recall: 0.6666666666666666
F1-Score: 0.6666666666666666

Classification Report:
              precision    recall  f1-score   support

           0       0.94      0.94      0.94        17
           1       0.67      0.67      0.67         3

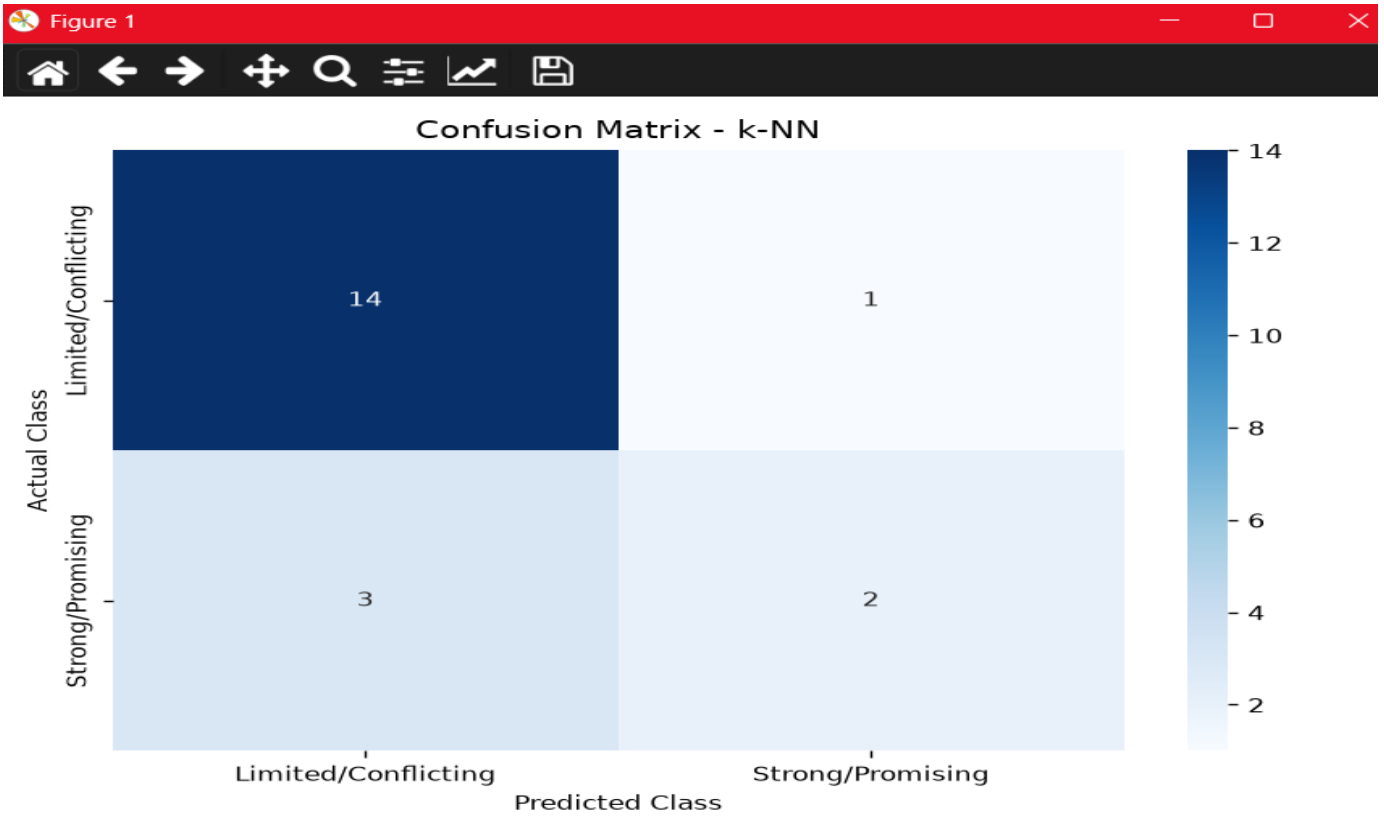
    accuracy                           0.90        20
   macro avg       0.80      0.80      0.80        20
weighted avg       0.90      0.90      0.90        20

Confusion matrix plot saved to: c:\Users\HP\OneDrive\Desktop\25 MCA CU\25MCA20133\3rd Sem 2026\25CAP-702 Machine Learning Lab\Experiment 3B\decision_tree_confusion_matrix.png
```



k-NN :

```
import os
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import (
confusion_matrix,
precision_score,
recall_score,
f1_score,
accuracy_score,
classification_report
)
import matplotlib.pyplot as plt
import seaborn as sns
csv_filename = "Sports Supplements.csv"
if os.path.exists(csv_filename):
csv_path = csv_filename
elif os.path.exists(os.path.join(os.path.dirname(os.path.abspath(__file__)), csv_filename)):
csv_path = os.path.join(os.path.dirname(os.path.abspath(__file__)), csv_filename)
elif os.path.exists("OneDrive/Desktop/Machine Learning/Sports Supplements.csv"):
csv_path = "OneDrive/Desktop/Machine Learning/Sports Supplements.csv"
else:
csv_path = r"C:\Users\HP\OneDrive\Desktop\Machine Learning\Sports Supplements.csv"
```



```
k-NN confusion matrix plot saved to: c:\Users\HP\OneDrive\Desktop\25 MCA CU\25MCA20133\3rd Sem 2026\25CAP-702 Machine
Learning Lab\Experiment 3B\knn_confusion_matrix.png
F1-Score: 0.5

Classification Report:
                    precision    recall  f1-score   support

Limited/Conflicting       0.82      0.93      0.88        15
   Strong/Promising       0.67      0.40      0.50         5

           accuracy                           0.80        20
          macro avg       0.75      0.67      0.69        20
       weighted avg       0.78      0.80      0.78        20


[Done] exited with code=0 in 131.018 seconds
```

- **Learning outcomes(B):**
  1. Implemented Decision Tree and k-NN classification algorithms on a suitable dataset.
  2. Evaluated model performance using confusion matrix, precision, recall, and F1-score.
  3. Compared the performance of Decision Tree and k-NN classifiers.
  4. Interpreted classification results and identify the better-performing model.